Machine Learning

Kaarlo

October 26, 2026

Contents

1 Lessons **1**
- 1.1 Motivation: what problem is machine learning solving? . . . 1
- 1.2 The supervised learning contract . . . 2
- 1.3 A layer is a matrix, a batch is a stacked matrix . . . 2
- 1.4 The objective is a scalar, so the gradient is a vector . . . 4
- 1.5 The full backward pass of a one-hidden-layer MLP . . . 6
- 1.6 Using the gradient: gradient descent . . . 7
- 1.7 What this lesson did not do . . . 8

2 Exercises **8**

3 Solutions **11**

4 Feedback **11**

1 Lessons

1.1 Motivation: what problem is machine learning solving?

Everything that follows is an answer to one question. We can measure a *loss*: a single number that says how badly some function f_θ performs on data. How do we find the parameters θ that make it small?

This reframes what you already know. "A neural network is $w \cdot x + b$" describes one *function* in a large *family* of functions. Machine learning is not about that function; it is about the optimisation problem sitting on top of it. Three ingredients are needed:

1. a family of candidate functions, called the *model* or *hypothesis class*, indexed by parameters θ;
2. a *loss* that turns "how wrong is f_θ on this example" into a real number;
3. an algorithm that searches θ-space for a low loss.

Matrices enter in step 1, because a linear layer evaluated on a batch of inputs is a matrix product. Backpropagation enters in step 3, because you cannot search $\mathbb{R}^d$ blindly when d is 10^9: you need the gradient, and backprop is the algorithm that computes it.

The roadmap for this topic:

1. supervised learning as loss minimisation, and the linear layer as a matrix (this lesson);
2. the chain rule in matrix form, and backpropagation as reverse-mode automatic differentiation (this lesson);

3. gradient descent, minibatching, and then generalisation—why small training loss is not the goal;

4. the same machinery for sequential decisions: dynamic programming, Q-learning, and the policy gradient, which is backprop applied to a stochastic objective.

1.2 The supervised learning contract

Definition 1.1 (Supervised learning problem)**.** An *example* is a pair $(\mathbf{x}, \mathbf{y})$ with $\mathbf{x} \in \mathbb{R}^n$ (the input or features) and $\mathbf{y} \in \mathbb{R}^k$ (the target). A *dataset* is a finite list $D = ((\mathbf{x}_1, \mathbf{y}_1), \dots, (\mathbf{x}_m, \mathbf{y}_m))$.

Fix a *model* $f_\theta \colon \mathbb{R}^n \to \mathbb{R}^k$ depending on parameters $\theta \in \mathbb{R}^d$, and a *per-example loss* $\ell \colon \mathbb{R}^k \times \mathbb{R}^k \to \mathbb{R}_{\geq 0}$. The *empirical risk* of θ is

$$L_D(\theta) = \frac{1}{m} \sum_{i=1}^{m} \ell\big(f_\theta(\mathbf{x}_i), \mathbf{y}_i\big).$$

Training means solving, at least approximately, $\min_\theta L_D(\theta)$.

Two comments on this definition, both of which matter later.

First, L_D is an *average*, not a sum. This is only a choice of scale, but it means the loss does not blow up when you change the batch size, and an "average gradient" is genuinely an average.

Second, L_D is a function of one variable: the flat vector θ. All the interesting structure (matrices, layers, activations) is hidden inside f_θ. This is why the mathematics of training is *calculus plus linear algebra*, and why we will never need a separate theory for each architecture: only $\partial L / \partial \theta$ is needed, however f_θ is built.

Example 1.2 (Losses you will meet)**.**

(a) *Squared error*, for regression: $\ell(\hat{\mathbf{y}}, \mathbf{y}) = \frac{1}{2}\|\hat{\mathbf{y}} - \mathbf{y}\|^2$. The factor $\frac{1}{2}$ cancels the 2 from differentiating a square; it is pure convenience and changes no minimiser.

(b) *Mean squared error* (MSE) over a batch: $L = \frac{1}{2m}\|Z - Y\|_F^2$, where the rows of $Z \in \mathbb{R}^{m \times k}$ are the predictions and the rows of Y are the targets, and $\|\cdot\|_F$ is the Frobenius norm $\|A\|_F^2 = \sum_{ij} A_{ij}^2$.

(c) *Softmax cross-entropy*, for C-way classification, which we will build in a later lesson from the same ingredients.

The Frobenius norm is worth noticing: it is the Euclidean norm of the matrix flattened into a vector. So squaring it is exactly "sum of squares over all entries", and MSE is "average squared coordinate error".

1.3 A layer is a matrix, a batch is a stacked matrix

Definition 1.3 (Affine (fully connected) layer)**.** An affine layer with n inputs and p outputs has parameters $W \in \mathbb{R}^{n \times p}$ (the *weight matrix*) and $\mathbf{b} \in \mathbb{R}^p$ (the *bias*). It maps a single input $\mathbf{x} \in \mathbb{R}^n$ to

$$\mathbf{z} = W^\top \mathbf{x} + \mathbf{b} \in \mathbb{R}^p.$$

Note the transposition: with $\mathbf{x}$ as a column, the j-th output coordinate is $z_j = \sum_{k=1}^{n} W_{kj} x_k + b_j$, i.e. the inner product of $\mathbf{x}$ with the j-th *column* of W. In your linear algebra course the layer would naturally be written as $A\mathbf{x} + \mathbf{b}$ with $A \in \mathbb{R}^{p \times n}$; the convention above stores $A^\top$ instead so that the batch version needs no transposition. Both are the same map; only the storage differs.

Now feed a whole batch at once. Put the inputs as the *rows* of $X \in \mathbb{R}^{m \times n}$ and the outputs as the rows of $Z \in \mathbb{R}^{m \times p}$. Then

$$Z = XW + \mathbf{1}_m \mathbf{b}^\top,$$

where $\mathbf{1}_m \in \mathbb{R}^m$ is the all-ones vector, so $\mathbf{1}_m \mathbf{b}^\top \in \mathbb{R}^{m \times p}$ has every row equal to $\mathbf{b}^\top$. Row i of this identity reads $\mathbf{z}_i^\top = \mathbf{x}_i^\top W + \mathbf{b}^\top$, which is the single-example layer transposed. This is the whole content of the next proposition.

Proposition 1.4 (Batching is not a new operation)**.** *Let $X \in \mathbb{R}^{m \times n}$ have rows $\mathbf{x}_1^\top, \dots, \mathbf{x}_m^\top$. Then the rows of $Z = XW + \mathbf{1}_m \mathbf{b}^\top$ are exactly $f(\mathbf{x}_1)^\top, \dots, f(\mathbf{x}_m)^\top$ for the affine layer f above. Consequently the batch computation is one matrix multiplication whose arithmetic cost is $\Theta(mnp)$, i.e. exactly m times the cost of a single example, with no saving from sharing the work.*

Proof. Row i of XW is $\sum_{k=1}^{n} X_{ik} W_{k\cdot} = \mathbf{x}_i^\top W$, and row i of $\mathbf{1}_m \mathbf{b}^\top$ is $\mathbf{b}^\top$. Add them. Independence across i holds because the i-th row of Z depends only on the i-th row of X. □

Remark (Why rows and not columns). Storing examples as rows is a convention, and PyTorch, JAX and TensorFlow all follow it. The concrete benefit is that the feature axis n is the *second* index of X, so the n entries belonging to one example are contiguous in a row-major array, and the whole layer is a single BLAS call XW with no transposition of the batch. The cost is conceptual: the elementary linear-algebra object "apply the map to $\mathbf{x}$" becomes "multiply the row by W", which is why the transpose in the definition of the layer cannot be avoided. Pick one convention, write it down, and check shapes against it; mixing the two is the source of most dimension errors.

Remark (Why a nonlinearity is mandatory). By theorem 1.4 a layer is affine. Compose two affine layers:

$$\mathbf{z} = W_2^\top \big(W_1^\top \mathbf{x} + \mathbf{b}_1\big) + \mathbf{b}_2 = (W_1 W_2)^\top \mathbf{x} + \big(W_2^\top \mathbf{b}_1 + \mathbf{b}_2\big).$$

The composition is again affine, with weight $W_1 W_2$. So depth without nonlinearity buys nothing: you are still fitting a single affine map, now with a rank constraint ($\operatorname{rank}(W_1 W_2) \le \min(\operatorname{rank} W_1, \operatorname{rank} W_2)$). This is the same "rank cannot increase under composition" argument as in your linear algebra exercises, and it is the reason every real network interleaves a nonlinear map.

Definition 1.5 (Activation, and an MLP)**.** An *activation* is a nonlinear scalar function σ applied entrywise to a matrix, written $\sigma(Z)$ with $\sigma(Z)_{ij} = \sigma(Z_{ij})$. The standard choice here is the rectified linear unit

$$\operatorname{relu}(t) = \max(t, 0).$$

A *multi-layer perceptron* (MLP) with one hidden layer is

$$Z_1 = XW_1 + \mathbf{1}\mathbf{b}_1^\top, \qquad R = \operatorname{relu}(Z_1), \qquad Z_2 = RW_2 + \mathbf{1}\mathbf{b}_2^\top,$$

with $W_1 \in \mathbb{R}^{n \times h}$, $W_2 \in \mathbb{R}^{h \times k}$. Z_1 is called the *pre-activation* and R the *hidden representation*.

Why does this help, given that each layer is affine? Entrywise relu applied to a matmul output is a *piecewise*-affine map. Fix the pattern of which entries are positive (a boolean matrix, the *mask*); on the region of inputs X where that pattern is constant, the network reduces to a single affine map, and different regions give different affine maps. So an MLP is an affine spline: many linear pieces glued together. The mask is a piece of data depending on the input, which is precisely what a fixed matrix cannot express.

Remark (Parameter counting). The first layer has $nh + h$ parameters, the second $hk + k$, total $nh + h + hk + k$. For $n = 784$, $h = 128$, $k = 10$ this is about 10^5; for language models the same formula with $h \sim 10^4$ in dozens of layers yields 10^9–10^{11}. Number the parameters in some fixed order and stack them into $\theta \in \mathbb{R}^d$. Gradients will have shape d too, and the whole point of backprop is to get all d of them for roughly the cost of one forward pass.

1.4 The objective is a scalar, so the gradient is a vector

Definition 1.6 (Gradient)**.** For a differentiable $L\colon \mathbb{R}^d \to \mathbb{R}$, the gradient $\nabla L(\theta)$ is the vector in $\mathbb{R}^d$ whose j-th entry is $\partial L/\partial \theta_j$. The *total derivative* at θ is the row vector $\nabla L(\theta)^\top$, the unique linear map $\mathbb{R}^d \to \mathbb{R}$ with

$$L(\theta + \delta) = L(\theta) + \nabla L(\theta)^\top \delta + o(\|\delta\|).$$

Everything downstream depends on one structural fact: *the loss is scalar.* Whatever the shape of an intermediate quantity, the derivative of L with respect to it has the same shape as the quantity. This is why we can avoid the 4-index tensors that "derivative of a matrix with respect to a matrix" would otherwise require.

Definition 1.7 (Adjoint)**.** Let L be the scalar loss and let G be any intermediate matrix or vector in the computation. Write

$$\bar{G} = \frac{\partial L}{\partial G}, \qquad \bar{G}_{ij} = \frac{\partial L}{\partial G_{ij}},$$

so $\bar{G}$ has the *same shape as* G. We call $\bar{G}$ the *adjoint* of G, also called its *upstream gradient.* For the parameters, $\bar{W}$ and $\bar{\mathbf{b}}$ are the gradient pieces we want.

The adjoint is not the naive "derivative matrix". If $G \in \mathbb{R}^{m\times p}$ then G has mp entries; a linear map $\mathbb{R}^{mp} \to \mathbb{R}$ is described by mp numbers, and $\bar{G}$ is exactly that description. In linear algebra language, $\bar{G}$ is the element of the dual space $(\mathbb{R}^{m\times p})^*$ corresponding to $\mathrm{d}L$, transported back to G's own shape. That identification is what makes "same shape" a legal convention.

Lemma 1.8 (Chain rule, adjoint form)**.** *Suppose L depends on G and G depends on H through a differentiable map Φ, so that $G = \Phi(H)$. Then the adjoints satisfy*

$$\bar{H} = \big(D\Phi(H)\big)^\top \bar{G},$$

where $D\Phi(H)$ is the Jacobian of Φ at H (a linear map between the flattened spaces) and $^\top$ denotes its transpose (the adjoint map).

Proof. Flatten H and G into vectors $\mathbf{h}, \mathbf{g}$. By definition $D\Phi$ is the matrix with $\partial g_a/\partial h_b$ in position (a, b). Then for each b,

$$\frac{\partial L}{\partial h_b} = \sum_a \frac{\partial L}{\partial g_a}\frac{\partial g_a}{\partial h_b} = \sum_a \big(D\Phi\big)_{ab}\bar{G}_a = \big((D\Phi)^\top \bar{G}\big)_b. \qquad \square$$

So backpropagation is one line of mathematics: *multiply the upstream adjoint by the transpose of the Jacobian, at every step, moving backwards.* The only work is computing $D\Phi^\top \bar{G}$ for each kind of layer without ever forming $D\Phi$. The saving is enormous. For a layer with $n = 10^3$ inputs, $p = 10^3$ outputs and a batch of $m = 10^4$ examples, the Jacobian of $W \mapsto Z$ is the $(mp) \times (np)$ matrix of size $10^7 \times 10^6$, while the product $X^\top \bar{Z}$ is an $n \times p$ matrix computed with one matmul. Reverse mode never materialises the Jacobian; it only ever applies its transpose to a vector.

Theorem 1.9 (Backprop through an affine layer)**.** *Let $Z = XW + \mathbf{1}_m \mathbf{b}^\top$ with $X \in \mathbb{R}^{m\times n}$, $W \in \mathbb{R}^{n\times p}$, $\mathbf{b} \in \mathbb{R}^p$, and let $\bar{Z}$ be the adjoint of Z. Then*

$$\bar{W} = X^\top \bar{Z}, \qquad \bar{\mathbf{b}} = \bar{Z}^\top \mathbf{1}_m, \qquad \bar{X} = \bar{Z}\, W^\top.$$

Proof. Since $Z_{ij} = \sum_k X_{ik} W_{kj} + b_j$, we have $\partial Z_{ij}/\partial W_{kj'} = \delta_{jj'} X_{ik}$, $\partial Z_{ij}/\partial b_{j'} = \delta_{jj'}$, and $\partial Z_{ij}/\partial X_{ik'} = \delta_{kk'} W_{kj}$. Summing over all (i, j) with the adjoint weights $\bar{Z}_{ij}$:

$$\bar{W}_{kj} = \sum_i \sum_{j'} \bar{Z}_{ij'} \frac{\partial Z_{ij'}}{\partial W_{kj}} = \sum_i \bar{Z}_{ij} X_{ik} = (X^\top \bar{Z})_{kj},$$

$$\bar{b}_j = \sum_i \sum_{j'} \bar{Z}_{ij'} \frac{\partial Z_{ij'}}{\partial b_j} = \sum_i \bar{Z}_{ij} = (\bar{Z}^\top \mathbf{1}_m)_j,$$

$$\bar{X}_{ik} = \sum_{i'} \sum_j \bar{Z}_{i'j} \frac{\partial Z_{i'j}}{\partial X_{ik}} = \sum_j \bar{Z}_{ij} W_{kj} = (\bar{Z} W^\top)_{ik}.$$ □

Remark (Reading the three formulas). All three are "sum over the index the layer contracted away".

- $\bar{W} = X^\top \bar{Z}$: sum the outer products $\mathbf{x}_i \bar{\mathbf{z}}_i^\top$ over the batch. Each example contributes the change it would like in W.
- $\bar{\mathbf{b}} = \bar{Z}^\top \mathbf{1}_m$: column sums of $\bar{Z}$. The bias is added to every example identically, so its gradient is the total over the batch.
- $\bar{X} = \bar{Z} W^\top$: the same map as the forward pass but with W transposed, pushing the error back to the inputs. If you do not need $\bar{X}$ (the first layer), skip it—this is the one step you can omit.

Shapes are a complete correctness check: $X^\top \bar{Z}$ is $(n \times m)(m \times p) = n \times p$, matching W; $\bar{Z} W^\top$ is $(m \times p)(p \times n) = m \times n$, matching X. A dimension mismatch always means you transposed the wrong factor.

Theorem 1.10 (Backprop through an entrywise activation)**.** *Let $R = \sigma(Z)$ with σ applied entrywise and differentiable. Then*

$$\bar{Z} = \bar{R} \odot \sigma'(Z),$$

where $\odot$ is the Hadamard (entrywise) product and $\sigma'(Z)$ is the matrix of entrywise derivatives. For $\sigma = \mathrm{relu}$, $\sigma'(t) = \mathbf{1}[t > 0]$, so

$$\bar{Z}_{ij} = \bar{R}_{ij} \cdot \mathbf{1}[Z_{ij} > 0].$$

Proof. Since R_{ij} depends only on Z_{ij}, and $R_{ij'}$ for $j' \neq j$ does not depend on Z_{ij}, the Jacobian is diagonal within each entry: $\partial R_{ij}/\partial Z_{ij} = \sigma'(Z_{ij})$ and $\partial R_{ij}/\partial Z_{i'j'} = 0$ if $(i, j) \neq (i', j')$. Hence $\bar{Z}_{ij} = \sum_{i'j'} \bar{R}_{i'j'} \partial R_{i'j'}/\partial Z_{ij} = \bar{R}_{ij} \sigma'(Z_{ij})$. □

Remark (The ReLU derivative at 0). The function relu is not differentiable at 0, and no derivative value there is "correct". Every framework picks 0, which means an exactly-zero pre-activation receives no gradient. This is a convention, not a theorem, and it is the usual way a unit dies (exercise 2.6). All our formulas use $\mathbf{1}[t > 0]$, matching that convention.

The ReLU adjoint is a mask multiply, and that is the whole story of "nonlinearity" in the backward pass: the forward pass decides which units are on, and the backward pass lets gradient through exactly those.

Example 1.11 (Adjunction is not optional: a wrong-but-plausible formula)**.** A common mistake is to write $\bar{Z} = \sigma'(\bar{R})$ or to forget that the mask must use the *forward* values Z, not $\bar{R}$. Theorem 1.10 shows both are wrong: the mask is a fixed 0/1 pattern determined in the forward pass, and it multiplies the incoming gradient entrywise. Similarly, writing $W_2 \bar{R}$ instead of $\bar{R} W_2^\top$ has incompatible shapes (unless $m = k$, and then it is silently wrong). *Check shapes before you check algebra.*

1.5 The full backward pass of a one-hidden-layer MLP

Assemble the pieces. With $Z_1 = XW_1 + \mathbf{1}\mathbf{b}_1^\top$, $R = \mathrm{relu}(Z_1)$, $Z_2 = RW_2 + \mathbf{1}\mathbf{b}_2^\top$, and the mean squared error objective

$$L = \frac{1}{2m}\|Z_2 - Y\|_F^2,$$

the adjoints are, in reverse order:

$$\bar{Z}_2 = \frac{1}{m}(Z_2 - Y), \tag{1}$$

$$\bar{W}_2 = R^\top \bar{Z}_2, \qquad \bar{\mathbf{b}}_2 = \bar{Z}_2^\top \mathbf{1}_m, \tag{2}$$

$$\bar{R} = \bar{Z}_2 W_2^\top, \tag{3}$$

$$\bar{Z}_1 = \bar{R} \odot \mathbf{1}[Z_1 > 0], \tag{4}$$

$$\bar{W}_1 = X^\top \bar{Z}_1, \qquad \bar{\mathbf{b}}_1 = \bar{Z}_1^\top \mathbf{1}_m. \tag{5}$$

The only ad hoc line is (1); it is a direct computation, which we do in the exercises. Everything else is theorem 1.9 and theorem 1.10.

Remark (Why this is fast). Let d be the number of parameters and T_{fwd} the cost of evaluating L. The backward pass costs about $2T_{\mathrm{fwd}}$: each layer theorem does one matrix multiply whose cost is comparable to the forward one. Crucially the cost is *independent of* d as a separate factor—it is proportional to the size of the computation, which already scales with d. A naive finite-difference gradient estimate, by contrast, needs one full forward pass per parameter: $O(d) \cdot T_{\mathrm{fwd}}$.

The structural reason is that L has one output. Forward-mode differentiation propagates a directional derivative $\langle \nabla L, \mathbf{v} \rangle$ (a *Jacobian-vector product*), which is cheap per input direction but must be repeated for each of the d parameters. Reverse-mode propagates $D\Phi^\top \bar{G}$ (a *vector-Jacobian product*), which is cheap per output. Since the output is one scalar, one reverse sweep suffices. This is the entire content of "backpropagation", and it is exactly reverse-mode automatic differentiation applied to the scalar loss.

Remark (Common mistakes).

(a) *Forgetting the* $1/m$. If the loss is the mean, every gradient carries $1/m$. Omit it and the effective learning rate scales with batch size, which is the single most common reason a training run "mysteriously" diverges when you change the batch size.

(b) *Transposing the wrong factor.* Use the shape check above. Every *weight* gradient has the form (the layer's input)$^\top \bar{Z}$: $\bar{W}_2 = R^\top \bar{Z}_2$ and $\bar{W}_1 = X^\top \bar{Z}_1$. The bias gradient is instead the column sum of $\bar{Z}$. For the adjoint passed *through* to the input, the transposed weight goes on the *right*: $\bar{R} = \bar{Z}_2 W_2^\top$, $\bar{X} = \bar{Z} W^\top$.

(c) *Accidentally differentiating through the data.* X and Y are constants. $\bar{X}$ is only needed when a layer sits below; forgetting to stop the backward pass at the first layer is harmless, but "updating" X is not.

(d) *Applying the mask in the wrong place.* The rule is $\bar{Z}_1 = \bar{R} \odot M$ with $M = \mathbf{1}[Z_1 > 0]$ the *forward* mask. Writing $\mathrm{relu}(\bar{R})$ is wrong: that is a different, nonlinear operation. Multiplying M into $\bar{W}_1$ entrywise is also wrong; the mask has to be applied to $\bar{R}$ *before* the matrix product $X^\top(\cdot)$.

(e) *Believing the gradient is the update.* The gradient is a direction of steepest increase; training moves *against* it. Also, a zero gradient component means "no first-order information", not "this parameter is useless".

1.6 Using the gradient: gradient descent

Definition 1.12 (Gradient descent)**.** With step size (learning rate) $\eta > 0$, the update is

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla L_D\big(\theta^{(t)}\big).$$

Stochastic gradient descent (SGD) replaces L_D by the average loss over a random *minibatch* of examples, which is an unbiased estimator of ∇L_D whenever the batch is sampled uniformly (see the exercises).

Proposition 1.13 (Descent lemma)**.** *Let L be differentiable at θ with $\nabla L(\theta) \neq 0$. Then for every sufficiently small $\eta > 0$, $L(\theta - \eta \nabla L(\theta)) < L(\theta)$.*

Proof. By the definition of the total derivative,

$$L(\theta - \eta \nabla L(\theta)) = L(\theta) - \eta \|\nabla L(\theta)\|^2 + o(\eta \|\nabla L(\theta)\|).$$

Write $r(\eta) = o(\eta\|\nabla L\|)/\eta \to 0$ as $\eta \to 0$. Then the difference equals $-\eta\big(\|\nabla L\|^2 - r(\eta)\big)$, which is negative once $|r(\eta)| < \|\nabla L\|^2$. □

This is a local statement only. It says a small enough step decreases the loss; it says nothing about reaching a global minimum, and nothing about the gap between L_D and performance on unseen data. Both are real issues, taken up in the next lesson.

Example 1.14 (A complete numerical backward pass)**.** Take $m = 3$ examples with $n = 2$ features, a hidden layer of width $h = 2$, and one output ($k = 1$):

$$X = \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & -1 \end{pmatrix}, \quad Y = \begin{pmatrix} 1 \\ 0 \\ 2 \end{pmatrix}, \quad W_1 = \begin{pmatrix} 1 & -2 \\ 2 & 1 \end{pmatrix}, \quad \mathbf{b}_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad W_2 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \quad b_2 = \tfrac{1}{2}.$$

Forward. $Z_1 = XW_1 + \mathbf{1}\mathbf{b}_1^\top$, so

$$Z_1 = \begin{pmatrix} 5 & 1 \\ 4 & -2 \\ -1 & -2 \end{pmatrix}, \qquad R = \operatorname{relu}(Z_1) = \begin{pmatrix} 5 & 1 \\ 4 & 0 \\ 0 & 0 \end{pmatrix},$$

and $Z_2 = RW_2 + b_2\mathbf{1}$ gives $Z_2 = (4.5,\, 4.5,\, 0.5)^\top$. Hence

$$L = \frac{1}{2 \cdot 3}\big(3.5^2 + 4.5^2 + (-1.5)^2\big) = \frac{34.75}{6} \approx 5.792.$$

Backward. Applying (1)–(5):

$$\bar{Z}_2 = \tfrac{1}{3}(3.5,\, 4.5,\, -1.5)^\top \approx (1.167,\, 1.5,\, -0.5)^\top,$$

$$\bar{W}_2 = R^\top \bar{Z}_2 = \begin{pmatrix} 5 \cdot 1.167 + 4 \cdot 1.5 \\ 1 \cdot 1.167 \end{pmatrix} \approx \begin{pmatrix} 11.833 \\ 1.167 \end{pmatrix}, \qquad \bar{b}_2 = \textstyle\sum_i \bar{Z}_{2,i} \approx 2.167,$$

$$\bar{R} = \bar{Z}_2 W_2^\top = \begin{pmatrix} 1.167 & -1.167 \\ 1.5 & -1.5 \\ -0.5 & 0.5 \end{pmatrix},$$

$$\bar{Z}_1 = \bar{R} \odot \mathbf{1}[Z_1 > 0] = \begin{pmatrix} 1.167 & -1.167 \\ 1.5 & 0 \\ 0 & 0 \end{pmatrix},$$

$$\bar{W}_1 = X^\top \bar{Z}_1 = \begin{pmatrix} 4.167 & -1.167 \\ 3.833 & -2.333 \end{pmatrix}, \qquad \bar{\mathbf{b}}_1 \approx \begin{pmatrix} 2.667 \\ -1.167 \end{pmatrix}.$$

Three of the six ReLU units are off (Z_1 has three nonpositive entries), and their adjoints are exactly 0: those entries of $\bar{W}_1$ receive no signal at all from this batch.

Remark (Reading the example as a program). The computation is a graph, and the backward pass is the same graph reversed with each node replaced by its adjoint rule:

$$X \longrightarrow Z_1 \longrightarrow R \longrightarrow Z_2 \longrightarrow L,$$

with $W_1, \mathbf{b}_1$ attached to Z_1 and W_2, b_2 attached to Z_2. A framework stores exactly the values needed by the mask ((4) needs Z_1) and frees the rest, which is why activation memory dominates training cost for large models. In code the layer is an object with two methods: `forward(inputs) -> outputs` returning the cache, and `backward(adjoint) -> (input_adjoint, param_grads)`. That is the whole interface; exercise 2.8 asks you to justify it.

1.7 What this lesson did not do

Two honest gaps. First, we derived gradients but not the conditions for a minimum: the derivative test, convexity, saddle points and the fact that in high dimension almost every stationary point is a saddle. Second, everything here is about L_D, the loss on data we have. Nothing above explains why minimising L_D makes the model useful on new data, and the answer is not calculus: it involves capacity, regularisation, and the bias–variance decomposition.

Both come next. After that, the RL thread starts: in reinforcement learning the loss is *random*—it depends on sampled trajectories—so the object to differentiate is an expectation, and the policy gradient theorem is exactly the chain rule of this lesson applied to that expectation. Keep theorem 1.8 in mind; it is the only tool the policy gradient needs.

2 Exercises

Exercise 2.1. A batch of $m = 4$ examples with $n = 3$ features is fed to a single affine layer followed by relu:

$$X = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 3 & 1 \\ -1 & 1 & 1 \\ 2 & 1 & 0 \end{pmatrix}, \qquad W = \begin{pmatrix} 1 & -1 \\ 0 & 2 \\ 1 & 1 \end{pmatrix}, \qquad \mathbf{b} = \begin{pmatrix} 0.5 \\ -0.5 \end{pmatrix}.$$

(a) Compute $Z = XW + \mathbf{1}_4\mathbf{b}^\top$ and $R = \mathrm{relu}(Z)$. Check that every row of Z equals $\mathbf{x}_i^\top W + \mathbf{b}^\top$ computed on its own.

(b) State the shapes of Z and R, and verify that $\mathbf{1}_4\mathbf{b}^\top$ really has the shape you claim.

(c) How many scalar parameters does this layer have, and what would it be for $n = 784$, $p = 128$?

(d) Which entries of R are zero, and why is the map $X \mapsto R$ not linear in X? Give explicit X and a nonzero scalar a for which $\mathrm{relu}(aX) \neq a\,\mathrm{relu}(X)$, and identify which entry of Z is responsible.

Exercise 2.2. Run a full backward pass by hand. The network has $m = n = 2$, $h = 2$, $k = 1$, and the mean squared error objective $L = \frac{1}{2m}\|Z_2 - Y\|_F^2$:

$$X = \begin{pmatrix} 1 & 2 \\ 3 & 1 \end{pmatrix}, \qquad Y = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \qquad W_1 = \begin{pmatrix} 0 & 1 \\ 1 & -1 \end{pmatrix}, \quad \mathbf{b}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad W_2 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad b_2 = 0.$$

(a) Compute Z_1, R, Z_2, and L.

(b) Compute $\bar{Z}_2$, $\bar{W}_2$, $\bar{b}_2$, $\bar{R}$, $\bar{Z}_1$, $\bar{W}_1$, $\bar{\mathbf{b}}_1$, using (1)–(5).

(c) Verify one entry of $\bar{W}_1$ numerically by the finite difference

$$\frac{L(W_1 + hE_{kj}) - L(W_1 - hE_{kj})}{2h}$$

with $h = 10^{-6}$, where E_{kj} is the elementary matrix with a single 1 in position (k, j).

(d) Explain why $\bar{Z}_1$ has a zero entry even though $\bar{R}$ does not.

Exercise 2.3. Let $Z \in \mathbb{R}^{m\times k}$ and $Y \in \mathbb{R}^{m\times k}$ be constant, and

$$L(Z) = \frac{1}{2m}\|Z - Y\|_F^2 = \frac{1}{2m}\sum_{i=1}^{m}\sum_{j=1}^{k}(Z_{ij} - Y_{ij})^2.$$

(a) Compute $\partial L/\partial Z_{ij}$ directly from the sum, and hence show that $\bar{Z} = \frac{1}{m}(Z - Y)$, i.e. that the adjoint of Z is $(Z - Y)/m$ and not its transpose.

(b) A classmate writes $\bar{Z} = (Z - Y)^\top/m$ "because the derivative of a vector with respect to a vector should be transposed". Give a concrete matrix Z and target Y for which the two expressions have different shapes, and say which shape the downstream formulas (2)–(3) require.

(c) Show that for $k = 1$ (scalar output per example) the formula reduces to the familiar 1-dimensional one: $\bar{Z}_i = (Z_i - Y_i)/m$.

Exercise 2.4. Fix a dataset with m examples, so that $L_D(\theta) = \frac{1}{m}\sum_{i=1}^{m}\ell_i(\theta)$ with $\ell_i(\theta) = \ell(f_\theta(\mathbf{x}_i), \mathbf{y}_i)$. A minibatch of size B is a subset $S \subseteq \{1, \dots, m\}$ of size B, chosen uniformly at random, and the minibatch loss is $\hat{L}_S(\theta) = \frac{1}{B}\sum_{i\in S}\ell_i(\theta)$.

(a) Show that $\mathbb{E}\big[\nabla\hat{L}_S(\theta)\big] = \nabla L_D(\theta)$ for every θ, where the expectation is over the uniform choice of S.

(b) Show that sampling with replacement is still unbiased: if $i_1, \dots, i_B$ are drawn independently and uniformly from $\{1, \dots, m\}$ and the batch loss is $\frac{1}{B}\sum_{b=1}^{B}\ell_{i_b}(\theta)$, then the expected gradient is again $\nabla L_D(\theta)$. Which of the two schemes (subset of size B, or B draws with replacement) has the smaller variance? Give an intuitive argument; an exact formula is not needed, but do say how the answer changes as B approaches m.

(c) Suppose someone uses the *sum* over the minibatch instead of the mean, i.e. $\tilde{L}_S = \sum_{i\in S}\ell_i$. Show that $\mathbb{E}[\nabla\tilde{L}_S] = B\nabla L_D(\theta)$, and explain in one sentence what this does to a fixed learning rate η when B is doubled.

Exercise 2.5. Consider a two-layer network with *no* activation between the layers:

$$Z_2 = (XW_1 + \mathbf{1}\mathbf{b}_1^\top)W_2 + \mathbf{1}\mathbf{b}_2^\top, \qquad W_1 \in \mathbb{R}^{n\times h},\ W_2 \in \mathbb{R}^{h\times k}.$$

Take $n = k = 3$, $h = 2$, and

$$W_1 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}, \qquad W_2 = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}, \qquad \mathbf{b}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad \mathbf{b}_2 = \mathbf{0}.$$

(a) Show that $Z_2 = XM + \mathbf{1}\mathbf{c}^\top$ for explicit M and $\mathbf{c}$, and give M and $\mathbf{c}$.

(b) Compute M, and compute $\operatorname{rank} M$ and $\operatorname{rank} W_1$. Then prove the general bound

$$\operatorname{rank}(W_1W_2) \le \min(\operatorname{rank} W_1, \operatorname{rank} W_2).$$

(c) Deduce that this network cannot compute the identity map $\mathbb{R}^3 \to \mathbb{R}^3$, no matter what W_1, W_2 are with $h = 2$, and state the smallest h that would be required if no nonlinearity were used.

(d) Explain in one or two sentences why inserting relu between the layers escapes the rank bound, even though each region of input space is still affine.

Exercise 2.6. Let $n = 1$, $h = 2$, $k = 1$, $m = 2$, with

$$X = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \qquad W_1 = \begin{pmatrix} -1 & -2 \end{pmatrix}, \qquad \mathbf{b}_1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \qquad W_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad b_2 = 0,$$

and the usual relu and mean squared error with targets $Y = (0, 1)^\top$.

(a) Compute Z_1, R, Z_2, and L.

(b) Compute all adjoints and show that $\bar{W}_1 = \mathbf{0}$ and $\bar{\mathbf{b}}_1 = \mathbf{0}$.

(c) Deduce exactly what one gradient descent step with $\eta > 0$ changes, and explain why no number of further steps can ever turn either hidden unit on. This failure mode is called a *dead ReLU*.

(d) Give one concrete change to the initialisation of W_1 or $\mathbf{b}_1$ that would avoid the dead unit here, and say precisely which quantity in your computation it makes nonzero.

Exercise 2.7. A *gradient check* compares an analytic adjoint $\bar{\theta}_j$ with the central finite difference

$$\hat{\theta}_j(h) = \frac{L(\theta + h\mathbf{e}_j) - L(\theta - h\mathbf{e}_j)}{2h}.$$

(a) Assuming L is three times differentiable, use Taylor expansion to show that

$$\hat{\theta}_j(h) = \frac{\partial L}{\partial \theta_j} + O(h^2).$$

(b) Explain why, in floating point arithmetic with relative precision $\varepsilon_{\text{mach}}$, the computed difference has an error term of order $\varepsilon_{\text{mach}}|L|/h$. Balancing this against the truncation error, derive the exponent p in $h \asymp \varepsilon_{\text{mach}}^{p}$ that minimises the total error, and give the numerical value of h for double precision. (Note that a *one-sided* difference $\big(L(\theta + h\mathbf{e}_j) - L(\theta)\big)/h$ has truncation error of order h, and therefore a different exponent. Say what it is.)

(c) Write pseudocode for `grad_check(loss, backward, theta)` that returns the maximum relative discrepancy over all d coordinates. State the precondition on the network and data. Then state one place where the check is expected to be inaccurate rather than wrong, and why.

(d) Your check passes for a two-layer network but fails for a deeper one. List two distinct causes that are consistent with a genuine implementation bug in the deeper network.

Exercise 2.8. A framework represents each layer as a pair of functions

`forward`: input $\mapsto$ (output, cache),
`backward`: (adjoint of output, cache) $\mapsto$ (adjoint of input, parameter adjoints).

Suppose the computation is a chain

$$G_0 \xrightarrow{\Phi_1} G_1 \xrightarrow{\Phi_2} \cdots \xrightarrow{\Phi_r} G_r,$$

with L a scalar function of G_r only.

(a) Prove by induction on r, using theorem 1.8, that applying the `backward` functions in the order $\Phi_r, \Phi_{r-1}, \ldots, \Phi_1$ yields $\bar{G}_1, \ldots, \bar{G}_r$ and, for each parameter of each Φ_a, its partial derivatives.

(b) Explain why the induction step needs the *cache* from the forward pass, and identify which lines of (1)–(5) would become impossible to evaluate without it.

(c) Suppose the same layer object is applied twice to the same input (weight sharing), so that $\Phi_1 = \Phi_2$ as functions of the same parameters. Explain why the backward pass of that layer runs twice and why the two results must be *added*, not overwritten, and connect this to the multivariate chain rule.

3 Solutions

4 Feedback